

TOMO I

BASH

Bourne Again Shell

Shell · Scripting · Automatización · Herramientas

Francisco Lagos · 2026

Tabla de Contenidos

[Historia de Bash](#)

[¿Qué es Bash?](#)

[¿Qué es una Shell?](#)

[Tipos de Shell](#)

[Primeros pasos en la terminal](#)

[Comandos básicos](#)

[Permisos en Linux](#)

[Administración de usuarios](#)

[Procesos](#)

[Redirecciones y tuberías](#)

[Variables de entorno y de Shell](#)

[Bash Scripting](#)

[Operaciones de comparación](#)

[Tipos de datos en Bash](#)

[Condicionales y bucles](#)

[Funciones](#)

[Arrays](#)

[Herramientas útiles en Bash](#)

[Expresiones Regulares](#)

[Automatización de tareas en Linux](#)

Historia de Bash

Bash fue creado por Brian Fox, quien comenzó a codificarlo el 10 de enero de 1988, después de que Richard Stallman se sintiera insatisfecho con la falta de progreso de un desarrollador anterior. Stallman y la Free Software Foundation (FSF) necesitaban un intérprete libre capaz de ejecutar scripts de shell existentes para un sistema completamente libre construido a partir de código BSD y GNU.

Fue creado como reemplazo de software libre del Bourne Shell (sh). Su nombre es un acrónimo y juego de palabras en inglés: "born-again" significa "nacido de nuevo".

¿Qué es Bash?

Bash (Bourne Again Shell) es un intérprete de comandos y lenguaje de scripting diseñado para sistemas tipo UNIX. Es el shell por defecto en la mayoría de distribuciones Linux y combina la compatibilidad con el Bourne Shell original con extensiones propias que facilitan tanto el uso interactivo como la escritura de scripts.

Nota: Ver también: ¿Qué es una Shell? para entender la diferencia entre shell e intérprete.

¿Qué es una Shell?

Una shell es un intérprete de comandos que actúa como intermediario entre el usuario y el kernel del sistema operativo. El usuario escribe un comando en texto, la shell lo interpreta y le indica al kernel qué ejecutar — ya sea llamando a un binario compilado como `/bin/lis`, o ejecutando directamente un built-in interno como `cd`.

Los binarios del sistema están distribuidos en directorios como `/bin`, `/usr/bin`, `/sbin` y `/usr/local/bin`. La shell es específicamente la interfaz de texto (CLI). La interfaz gráfica (GUI) es una capa distinta que no forma parte de la shell.

Tipos de Shell

Existen varios tipos de shell, cada uno con características distintas. Los más comunes en sistemas UNIX/Linux son Bash, Zsh y Fish, aunque también existen opciones minimalistas como Dash o multiplataforma como PowerShell.

Shell	Origen	Característica principal
Bash	GNU, 1989	Predeterminado en la mayoría de distribuciones Linux
Zsh	1990	Autocompletado avanzado, altamente configurable
Fish	2005	Sintaxis amigable, sugerencias automáticas integradas
Dash	1997	Minimalista, rápido, usado para scripts del sistema
PowerShell	Microsoft, 2006	Orientado a objetos, compatible con Windows y multiplataforma
sh (Bourne Shell)	AT&T, 1979	El intérprete original, base de los demás

Primeros pasos en la terminal

En Linux, los comandos siguen una estructura fija. Conocer esta estructura permite entender cualquier comando, incluso sin haberlo visto antes.

```
comando [opciones] [argumentos]

comando      - qué ejecutar
opciones     - modifican el comportamiento (van con - o --)
argumentos   - sobre qué actúa el comando

Ejemplo:
ls -la /home
ls           - comando
-la         - opciones (lista larga + archivos ocultos)
/home       - argumento
```

Comandos básicos

Navegación

Comando	Significado	Descripción
pwd	Print Working Directory	Muestra la ruta del directorio actual
ls	List	Lista archivos y directorios
cd	Change Directory	Cambia de directorio
tree	Tree	Muestra la estructura de directorios en forma de árbol

Manejo de directorios

Comando	Significado	Descripción
mkdir	Make Directory	Crea un directorio
rmdir	Remove Directory	Elimina un directorio vacío

Comando	Significado	Descripción
rm -r	Remove Recursive	Elimina un directorio y su contenido

Creación y gestión de archivos

Comando	Significado	Descripción
touch	Touch	Crea un archivo vacío o actualiza su fecha de modificación
cat	Concatenate	Muestra el contenido de un archivo
less	Less	Muestra el contenido paginado, permite scroll
more	More	Similar a less, más limitado
head	Head	Muestra las primeras líneas de un archivo
tail	Tail	Muestra las últimas líneas de un archivo
nano	Nano	Editor de texto simple en terminal
vim	Vi Improved	Editor de texto avanzado en terminal
file	File	Muestra el tipo de un archivo

Copiar, mover y renombrar

Comando	Significado	Descripción
cp	Copy	Copia archivos o directorios
mv	Move	Mueve o renombra archivos y directorios
rsync	Remote Sync	Sincroniza archivos local o remotamente

Búsqueda

Comando	Significado	Descripción
find	Find	Busca archivos y directorios en el sistema
locate	Locate	Busca archivos usando una base de datos indexada

Comando	Significado	Descripción
which	Which	Muestra la ruta del binario de un comando
whereis	Where Is	Muestra la ruta del binario, fuente y manual
grep	Global Regular Expression Print	Busca texto dentro de archivos

Información del sistema

Comando	Significado	Descripción
uname	Unix Name	Muestra información del sistema operativo
whoami	Who Am I	Muestra el usuario actual
id	ID	Muestra el UID, GID y grupos del usuario
hostname	Hostname	Muestra o cambia el nombre del equipo
uptime	Uptime	Muestra el tiempo que lleva encendido el sistema
df	Disk Free	Muestra el espacio libre en disco
du	Disk Usage	Muestra el espacio usado por archivos y directorios
free	Free	Muestra el uso de memoria RAM y swap
top	Top	Muestra procesos en tiempo real
htop	Human Top	Versión mejorada e interactiva de top

Extra útil

Comando	Significado	Descripción
man	Manual	Muestra el manual de un comando
clear	Clear	Limpiar la pantalla de la terminal
history	History	Muestra el historial de comandos ejecutados
echo	Echo	Imprime texto en la terminal
alias	Alias	Crea un atajo para un comando
exit	Exit	Cierra la sesión o terminal actual

Permisos en Linux

En Linux, cada archivo y directorio tiene permisos que controlan quién puede leerlo, modificarlo o ejecutarlo. Los permisos se asignan a tres entidades: el propietario del archivo, el grupo y el resto de usuarios.

Tipos de permiso

Permiso	Símbolo	Valor	Descripción
Lectura	r	4	Ver el contenido del archivo o directorio
Escritura	w	2	Modificar el contenido
Ejecución	x	1	Ejecutar el archivo o acceder al directorio

Las tres entidades

Entidad	Símbolo	Descripción
Propietario	u	Usuario dueño del archivo
Grupo	g	Grupo al que pertenece el archivo
Otros	o	El resto de usuarios del sistema

Cómo se ven los permisos

```
ls -l
-rwxr-xr-- 1 francisco devs 1234 Apr 29 archivo.sh

Se lee así:
- rwx r-x r--
|  |  |  |
|  |  |  |  otros: solo lectura
|  |  |  |  grupo: lectura y ejecución
|  |  |  |  propietario: lectura, escritura y ejecución
|  |  |  |  tipo: - archivo d directorio l enlace simbólico
```

Significado de cada campo

Campo	Valor	Descripción
Tipo y permisos	-rwxr-xr--	Tipo de archivo y permisos de cada entidad
Enlaces	1	Número de enlaces duros (hard links)
Propietario	francisco	Usuario dueño del archivo
Grupo	devs	Grupo dueño del archivo
Tamaño	1234	Tamaño en bytes
Fecha	Apr 29	Fecha de última modificación
Nombre	archivo.sh	Nombre del archivo

Notación octal

Los permisos se pueden expresar con números. Cada permiso tiene un valor: r=4, w=2, x=1. Se suman por entidad para obtener el número octal.

Octal	Permisos	Significado
7	rwx	Lectura, escritura y ejecución
6	rw-	Lectura y escritura
5	r-x	Lectura y ejecución
4	r--	Solo lectura
0	---	Sin permisos

```
chmod 754 archivo.sh
# propietario: 7 (rwx) grupo: 5 (r-x) otros: 4 (r--)
```

chmod — Change Mode

```
# Notación octal
chmod 755 archivo.sh      # propietario: rwx, grupo: r-x, otros: r-x
chmod 644 archivo.txt     # propietario: rw-, grupo: r--, otros: r--

# Notación simbólica
chmod u+x archivo.sh      # agrega ejecución al propietario
chmod g-w archivo.txt     # quita escritura al grupo
chmod o+r archivo.txt     # agrega lectura a otros
```

```
chmod a+x archivo.sh      # agrega ejecución a todos (a = all)
```

chown y chgrp

Al cambiar propietario o grupo solo se especifica el nuevo valor — el anterior se sobrescribe automáticamente.

```
# chown - Change Owner
chown francisco archivo.txt      # cambia propietario
chown francisco:devs archivo.txt  # cambia propietario y grupo
chown -R francisco /var/www      # recursivo

# chgrp - Change Group
chgrp devs archivo.txt
chgrp -R devs /var/www
```

umask

umask (user file creation mask) define los permisos que se restan por defecto al crear nuevos archivos y directorios.

```
umask      # muestra el umask actual (ej: 0022)
umask 027   # propietario: todo, grupo: r-x, otros: sin permisos

# El umask se resta de 666 (archivos) y 777 (directorios):
777 - 022 = 755 → directorios nuevos
666 - 022 = 644 → archivos nuevos
```

Administración de usuarios

En Linux, cada usuario tiene una identidad única con la que el sistema controla el acceso a recursos. Existen usuarios normales, el usuario root (superusuario con acceso total) y usuarios del sistema creados por servicios. La administración de usuarios es clave para la seguridad.

Archivos importantes

Archivo	Descripción
/etc/passwd	Lista de usuarios. Contiene nombre, UID, GID, directorio home y shell
/etc/shadow	Contraseñas encriptadas y políticas de expiración
/etc/group	Lista de grupos del sistema y sus miembros
/etc/sudoers	Define qué usuarios y grupos pueden ejecutar comandos como root

Comandos para usuarios

Comando	Significado	Descripción
useradd	User Add	Crea un nuevo usuario
usermod	User Modify	Modifica un usuario existente
userdel	User Delete	Elimina un usuario
passwd	Password	Cambia la contraseña de un usuario
whoami	Who Am I	Muestra el usuario actual
id	ID	Muestra UID, GID y grupos del usuario
su	Switch User	Cambia de usuario en la sesión actual
sudo	Superuser Do	Ejecuta un comando como root

Comandos para grupos

Comando	Significado	Descripción
groupadd	Group Add	Crea un nuevo grupo
groupmod	Group Modify	Modifica un grupo existente
groupdel	Group Delete	Elimina un grupo
groups	Groups	Muestra los grupos del usuario actual
gpasswd	Group Password	Administra miembros de un grupo
newgrp	New Group	Cambia el grupo activo temporalmente

Grupos y archivos

Al crear un archivo, se hereda el grupo principal del usuario que lo creó. El grupo principal es el primero que aparece al ejecutar `id`.

```
id
# uid=1000(francisco) gid=1000(francisco) groups=1000(francisco),1001(devs)
#
#
# grupo principal

# Cambiar grupo activo temporalmente
newgrp devs
touch archivo.txt    # el archivo queda con grupo devs

# Ver grupo de un archivo
ls -l archivo.txt
# -rw-r--r-- 1 francisco devs 0 Apr 29 archivo.txt
```

Ejemplos

```
useradd -m -s /bin/bash francisco # crear con home y shell bash
passwd francisco                  # asignar contraseña
usermod -aG devs francisco        # agregar al grupo devs
usermod -s /bin/zsh francisco     # cambiar shell
userdel -r francisco              # eliminar usuario y home

groupadd devs
su francisco                      # cambiar a usuario
```

```
su - # cambiar a root
sudo apt update
```

Procesos

En Linux, cada programa en ejecución es un proceso con un identificador único llamado PID (Process ID). El sistema operativo gestiona estos procesos asignándoles recursos como CPU y memoria.

Comandos de procesos

Comando	Significado	Descripción
ps	Process Status	Muestra los procesos activos del usuario actual
ps aux	Process Status All	Muestra todos los procesos del sistema
top	Top	Muestra procesos en tiempo real
htop	Human Top	Versión mejorada e interactiva de top
kill	Kill	Termina un proceso por su PID
killall	Kill All	Termina todos los procesos por nombre
pkill	Process Kill	Termina procesos por nombre o patrón
bg	Background	Envía un proceso al segundo plano
fg	Foreground	Trae un proceso al primer plano
jobs	Jobs	Lista los procesos en segundo plano
nice	Nice	Inicia un proceso con prioridad definida
renice	Renice	Cambia la prioridad de un proceso en ejecución
nohup	No Hang Up	Ejecuta un proceso que sobrevive al cerrar la terminal
pgrep	Process Grep	Busca procesos por nombre y devuelve su PID
pstree	Process Tree	Muestra los procesos en forma de árbol

Señales con kill

Señal	Número	Descripción
SIGTERM	15	Termina el proceso de forma ordenada (por defecto)
SIGKILL	9	Termina el proceso de forma forzada e inmediata
SIGHUP	1	Reinicia el proceso
SIGSTOP	19	Pausa el proceso
SIGCONT	18	Reanuda un proceso pausado

Ejemplos

```
ps aux | grep firefox      # buscar proceso
kill 1234                  # termina por PID
kill -9 1234               # termina forzado
killall firefox

comando &                  # ejecuta en segundo plano
Ctrl + Z                  # pausa el proceso actual
bg                          # envía al segundo plano
fg                          # trae al primer plano

nice -n 10 comando         # prioridad baja
renice -n 5 -p 1234
nohup comando &            # sobrevive al cerrar terminal
```

Redirecciones y tuberías

La salida de un comando puede redirigirse a un archivo o enviarse como entrada a otro comando. Esto permite encadenar comandos para construir operaciones complejas sin necesidad de scripts.

Entradas y salidas estándar

Nombre	Número	Descripción
stdin	0	Entrada estándar (teclado)
stdout	1	Salida estándar (pantalla)
stderr	2	Salida de errores (pantalla)

Operadores

Operador	Descripción
>	Redirige stdout a un archivo (sobreescribe)
>>	Redirige stdout a un archivo (agrega al final)
<	Toma stdin desde un archivo
2>	Redirige stderr a un archivo
2>>	Agrega stderr a un archivo
&>	Redirige stdout y stderr a un archivo
2>&1	Redirige stderr hacia donde va stdout
	Envía stdout de un comando como stdin del siguiente (pipe)
&	Envía stdout y stderr al siguiente comando

/dev/null

/dev/null es un archivo especial que descarta todo lo que recibe. Se usa para silenciar salidas que no interesan.

```
comando 2> /dev/null      # descarta errores
comando &> /dev/null      # descarta stdout y stderr
comando > /dev/null 2>&1  # forma alternativa equivalente

# Ver stdout y guardar errores en archivo
comando 2> errores.txt

# Guardar stdout y ver errores en pantalla
comando > salida.txt

# Separar stdout y stderr
comando > salida.txt 2> errores.txt
```

Comandos útiles con tuberías

Comando	Descripción
grep	Filtra líneas por patrón
sort	Ordena líneas
uniq	Elimina líneas duplicadas consecutivas
wc	Cuenta líneas, palabras o caracteres
cut	Extrae columnas de texto
awk	Procesa y filtra texto por columnas
sed	Edita texto en flujo
tee	Guarda stdout en archivo y lo muestra en pantalla
xargs	Convierte stdin en argumentos de otro comando

Variables de entorno y de Shell

En Linux existen dos tipos de variables: las variables de shell (también llamadas variables de usuario), que solo existen en la sesión actual y no son heredadas por los procesos hijos; y las variables de entorno, que están preconfiguradas en el sistema y son accesibles por todos los procesos hijos.

Variables de entorno comunes

Variable	Descripción
\$HOME	Directorio home del usuario
\$USER	Nombre del usuario actual
\$PATH	Rutas donde bash busca los binarios de comandos
\$SHELL	Shell actual en uso
\$PWD	Directorio de trabajo actual
\$OLDPWD	Directorio anterior
\$EDITOR	Editor de texto por defecto
\$LANG	Idioma y localización del sistema
\$PS1	Define el aspecto del prompt de la terminal

Comandos

Comando	Descripción
env	Muestra todas las variables de entorno activas
printenv	Muestra el valor de una variable específica
set	Muestra todas las variables de shell y de entorno
unset	Elimina una variable
export	Convierte una variable de shell en variable de entorno

Ejemplos y diferencia clave

```
# Crear variable de shell (solo sesión actual)
nombre="francisco"
bash -c 'echo $nombre'      # no muestra nada

# Exportar como variable de entorno
export nombre="francisco"
bash -c 'echo $nombre'      # muestra: francisco

# Agregar ruta al PATH
export PATH=$PATH:/home/francisco/scripts

# Eliminar variable
unset nombre
```

Variables permanentes

Las variables exportadas en terminal duran solo la sesión. Para hacerlas permanentes se agregan a un archivo de configuración.

Archivo	Alcance
~/.bashrc	Usuario actual, sesiones interactivas
~/.bash_profile	Usuario actual, sesiones de login
/etc/environment	Todos los usuarios del sistema

```
echo 'export EDITOR="vim"' >> ~/.bashrc
source ~/.bashrc      # aplicar cambios sin reiniciar
```

Variables especiales

Variable	Descripción
\$0	Nombre del script
\$1, \$2...	Argumentos pasados al script
\$#	Número de argumentos recibidos
\$@	Todos los argumentos como lista
\$?	Código de salida del último comando (0 = éxito)
\$\$	PID del script actual

Bash Scripting

Un script de bash es un archivo de texto que contiene una secuencia de comandos que se ejecutan en orden. Permite automatizar tareas repetitivas, combinar comandos y agregar lógica como condiciones y bucles.

Crear y ejecutar

```
touch script.sh          # crear
nano script.sh           # editar
chmod +x script.sh       # dar permisos de ejecución

./script.sh              # ejecutar desde directorio actual
bash script.sh            # sin permisos de ejecución
/ruta/completa/script.sh # con ruta absoluta
```

Ejecutar script en segundo plano

```
# Con & al final - se detiene si cierras la terminal
./script.sh &

# Con nohup - sobrevive al cerrar la terminal
nohup ./script.sh &

# Con nohup y redirigiendo la salida
nohup ./script.sh > /var/log/script.log 2>&1 &

# Ver que está corriendo
jobs          # procesos en segundo plano de la sesión actual
ps aux | grep script.sh

# Traerlo al primer plano si lo necesitas
fg %1

Con jobs se ven todos los procesos en segundo plano con su número
jobs
# [1]  Running    nohup ./backup.sh &
# [2]  Running    nohup ./monitoreo.sh &
# [3]  Stopped    ./otro.sh
```

```
fg %1 # trae backup.sh al primer plano
fg %2 # trae monitoreo.sh al primer plano
fg %3 # trae otro.sh al primer plano

También se puede usar el PID

ps aux | grep script      # ver los PIDs
fg %1                     # por número de job (más común)
```

Shebang (#!/bin/bash)

Le indica al sistema qué intérprete usar. Siempre va en la primera línea.

```
#!/bin/bash      # usa bash
#!/bin/sh        # usa sh (más portable)
#!/usr/bin/python3 # usa python3
```

Buenas prácticas

Las buenas prácticas ayudan a escribir scripts más robustos, seguros y fáciles de depurar.

```
#!/bin/bash
set -euo pipefail
# -e      detiene si un comando falla
# -u      error si se usa variable no definida
# -o pipefail detecta errores dentro de tuberías

# set -x activa modo debug (muestra cada comando antes de ejecutarlo)
set -x
comando1
set +x      # desactivar debug

# Usar llaves para evitar ambigüedad
echo "${nombre}_backup"      # correcto
echo "$nombre_backup"        # bash busca la variable $nombre_backup

# Comillas dobles para manejar espacios
rm "$archivo"                # correcto
rm $archivo                   # error si el nombre tiene espacios

# Verificar argumentos
if [ $# -eq 0 ]; then
    echo "Uso: $0 <argumento>"
    exit 1
fi
```

Operaciones de comparación

Permiten evaluar relaciones entre números, cadenas o archivos dentro de estructuras condicionales, devolviendo verdadero o falso.

Números

Operador	Inglés	Español
-eq	equal	igual
-ne	not equal	distinto
-gt	greater than	mayor que
-lt	less than	menor que
-ge	greater or equal	mayor o igual
-le	less or equal	menor o igual

Strings

Operador	Inglés	Descripción
=	equal	Cadenas iguales
!=	not equal	Cadenas distintas
-z	zero length	Cadena vacía
-n	non-zero length	Cadena no vacía
<	less than	Menor alfabéticamente
>	greater than	Mayor alfabéticamente

Archivos

Operador	Inglés	Descripción
-f	file	Es un archivo regular
-d	directory	Es un directorio
-e	exists	Existe (archivo o directorio)
-r	readable	Tiene permiso de lectura
-w	writable	Tiene permiso de escritura
-x	executable	Tiene permiso de ejecución
-s	size	Archivo no vacío
-L	link	Es un enlace simbólico

Tipos de datos en Bash

Bash maneja todos los valores como texto (strings) por defecto. No tiene tipos de datos estrictos como Python o C++, pero permite operar con enteros, simular decimales con `bc` y manipular strings con operadores propios.

Resumen de tipos

Tipo	Soporte nativo	Herramienta
String	Sí	Operadores <code>\${ }</code>
Integer	Sí	<code>\$(())</code> , <code>declare -i</code>
Float	No	<code>bc</code>
Boolean	No	Se simula con 0 y 1
Array indexado	Sí	<code>declare -a</code>
Array asociativo (dict)	Sí	<code>declare -A</code>

Strings

```

nombre="francisco"
frase='hola mundo'           # comillas simples: no interpreta variables
frase="hola $nombre"         # comillas dobles: interpreta variables

echo ${#nombre}              # longitud: 9
completo="${nombre} lagos"    # concatenar
echo ${nombre^^}             # FRANCISCO
echo ${nombre,,}             # francisco
echo ${nombre^}              # Francisco

# Substring
texto="hola mundo"
echo ${texto:0:4}             # hola
echo ${texto:5}              # mundo

# Reemplazar
echo ${texto/mundo/bash}     # hola bash
echo ${texto//o/0}           # h0la mund0

# Eliminar patrón
echo ${archivo%.txt}         # elimina sufijo .txt
echo ${archivo#archivo.}     # elimina prefijo archivo.

```

Integers y Floats

```
# Integers con $(( ))
echo $((10 + 3))      # 13
echo $((10 / 3))      # 3 (división entera)
echo $((10 % 3))      # 1 (módulo)
echo $((2 ** 8))      # 256 (potencia)

# Incremento/decremento
((a++))   ((a--))
((a+=3))  ((a*=2))

# Floats con bc
echo "scale=2; 5 / 2" | bc          # 2.50
pi=$(echo "scale=10; 22 / 7" | bc)

# Comparar decimales
resultado=$(echo "3.5 > 2.1" | bc)
if [ $resultado -eq 1 ]; then echo "mayor"; fi
```

declare

Opción	Descripción
declare -i	Entero
declare -r	Solo lectura (constante)
declare -l	Convierte a minúsculas automáticamente
declare -u	Convierte a mayúsculas automáticamente
declare -a	Array indexado
declare -A	Array asociativo

Entrada del usuario

```
read -p "¿Cuál es tu nombre? " nombre
echo "Hola $nombre"
```

Condicionales y bucles

Los condicionales ejecutan código según si una condición es verdadera o falsa. Los bucles repiten un bloque mientras se cumpla una condición o para cada elemento de una lista.

if / elif / else

```
#!/bin/bash
edad=20

if [ $edad -ge 18 ]; then
    echo "Mayor de edad"
elif [ $edad -eq 17 ]; then
    echo "Casi"
else
    echo "Menor de edad"
fi
```

Case (switch)

```
read -p "Opción: " opcion

case $opcion in
    1) echo "Elegiste 1" ;;
    2) echo "Elegiste 2" ;;
    *) echo "Opción inválida" ;;
esac
```

for

```
# Lista explícita
for i in 1 2 3 4 5; do
    echo "Número: $i"
done

# Rango
for i in {1..10}; do echo $i; done

# Archivos
for archivo in *.txt; do
    echo "Procesando: $archivo"
done

# Estilo C
for ((i=0; i<5; i++)); do echo $i; done
```

while y until

```
# while — repite mientras la condición sea verdadera
contador=0
while [ $contador -lt 5 ]; do
    echo "Contador: $contador"
    contador=$((contador + 1))
done

# until — repite hasta que la condición sea verdadera
until [ $contador -eq 5 ]; do
    echo $contador
    contador=$((contador + 1))
done
```

Funciones

Las funciones agrupan comandos bajo un nombre reutilizable. Pueden recibir argumentos, retornar valores y organizarse en archivos separados. Las funciones siempre son globales dentro del script donde se definen — no existen funciones locales.

Sintaxis y argumentos

```
# Declarar función
saludar() {
    echo "Hola $1, tienes $2 años"
}

saludar "francisco" 25
# Hola francisco, tienes 25 años
```

Variables locales

```
# Sin local: modifica la variable global
sin_local() { nombre="pedro"; }

# Con local: solo existe dentro de la función
con_local() { local nombre="pedro"; }

nombre="francisco"
sin_local
echo $nombre      # pedro — fue modificada

nombre="francisco"
con_local
echo $nombre      # francisco — no fue modificada
```

Retornar valores

```
# Con return — solo códigos 0-255
es_mayor() {
    [ $1 -ge 18 ] && return 0 || return 1
}
es_mayor 20 && echo "Mayor de edad"

# Con echo — retorna texto, se captura con $( )
cuadrado() { echo $(( $1 * $1 )); }
resultado=$(cuadrado 5)
echo $resultado      # 25
```

Ejemplo completo

Combina argumentos, variables locales, retorno con echo/\$() y llamada entre funciones:

```
#!/bin/bash

calcular_descuento() {
    local precio=$1
    local pct=$2
    echo $(echo "scale=2; $precio - ($precio * $pct / 100)" | bc)
}

aplicar_descuento() {
    local producto=$1
    local precio=$2
    local descuento=$3
    local final=$(calcular_descuento $precio $descuento)
    echo "$producto: $precio -> $final ($descuento% off)"
}

aplicar_descuento "Laptop" 1000 15
# Laptop: 1000 -> 850.00 (15% off)
```

Llamar funciones de otro archivo

```
# funciones.sh
saludar() { echo "Hola $1"; }

# script.sh
FUNCIONES="./funciones.sh"
if [ ! -f "$FUNCIONES" ]; then
    echo "Error: no se encontró $FUNCIONES"; exit 1
fi
source "$FUNCIONES"
saludar "francisco"
```

Arrays

Bash soporta dos tipos: arrays indexados (listas) y arrays asociativos (diccionarios). Permiten almacenar múltiples valores en una sola variable y recorrerlos con bucles.

Arrays indexados

```
frutas=("manzana" "pera" "uva")

echo ${frutas[0]}      # manzana
echo ${frutas[@]}      # todos los elementos
echo ${#frutas[@]}     # número de elementos: 3
echo ${!frutas[@]}     # índices: 0 1 2

frutas[0]="naranja"    # modificar
frutas+=("mango")      # agregar al final
unset frutas[1]        # eliminar elemento

# Recorrer
for fruta in "${frutas[@]}"; do echo $fruta; done

# Recorrer con índice
for i in "${!frutas[@]}"; do
    echo "$i: ${frutas[$i]}"
done

# Slicing
echo ${frutas[@]:1:3}   # desde índice 1, 3 elementos
```

Arrays asociativos (diccionarios)

Los arrays asociativos usan strings como claves en lugar de números, equivalentes a los diccionarios en Python. Requieren declare -A obligatoriamente y no garantizan un orden fijo al recorrerlos.

```
# declare -A es obligatorio
declare -A usuario
usuario["nombre"]="francisco"
usuario["edad"]=25
usuario["shell"]="bash"

echo ${usuario["nombre"]} # francisco
echo ${!usuario[@]}       # todas las claves
echo ${usuario[@]}        # todos los valores
echo ${#usuario[@]}       # número de elementos
```

```
usuario["edad"]=26          # modificar
usuario["ciudad"]="santiago" # nueva clave
unset usuario["shell"]      # eliminar clave

# Recorrer
for clave in "${!usuario[@]}; do
    echo "$clave: ${usuario[$clave]}"
done
```

Diferencias clave

	Array indexado	Array asociativo
Índice	Numérico (0, 1, 2...)	String ("nombre", "edad")
Declaración	Opcional: declare -a	Obligatorio: declare -A
Orden	Mantiene el orden	No garantiza orden
Equivalente Python	list	dict

Herramientas útiles en Bash

Linux ofrece herramientas de línea de comandos para buscar, filtrar y transformar texto. Son especialmente poderosas cuando se combinan con tuberías.

grep

```
grep "patron" archivo.txt
grep -i "patron" archivo.txt      # ignora mayúsculas/minúsculas
grep -r "patron" /directorio      # recursivo
grep -n "patron" archivo.txt      # muestra número de línea
grep -v "patron" archivo.txt      # líneas que NO coinciden
grep -c "patron" archivo.txt      # cuenta coincidencias
grep -w "patron" archivo.txt      # palabra completa
grep -A 2 "patron" archivo.txt    # 2 líneas después
grep -B 2 "patron" archivo.txt    # 2 líneas antes
grep -E "pat1|pat2"               # expresiones extendidas
```

find

find con -exec ejecuta un comando sobre cada archivo encontrado. {} representa el archivo encontrado y \; termina el -exec. Usando + en lugar de \; ejecuta el comando una sola vez con todos los archivos (más eficiente).

```
find /ruta -name "*.txt"
find /ruta -type f              # solo archivos
find /ruta -type d              # solo directorios
find /ruta -size +100M          # mayores a 100MB
find /ruta -mtime -7            # modificados últimos 7 días
find /ruta -empty               # vacíos

# -exec: {} = archivo encontrado \; = fin del comando
find . -name "*.sh" -exec chmod +x {} \;
find /var/log -name "*.log" -mtime +7 -exec rm {} \;

# Con + ejecuta el comando una sola vez con todos los archivos
find . -name "*.txt" -exec cat {} +
```

sed

sed (Stream EDitor) en Bash es un editor de texto no interactivo que transforma flujos de datos o archivos línea por línea.

```
sed 's/viejo/nuevo/' archivo.txt      # primera ocurrencia
sed 's/viejo/nuevo/g' archivo.txt    # todas las ocurrencias
sed -i 's/viejo/nuevo/g' archivo.txt  # edita el archivo directamente
sed '/patron/d' archivo.txt           # elimina líneas con patrón
sed '/^$/d' archivo.txt               # elimina líneas vacías
sed -n '1,5p' archivo.txt             # muestra líneas 1 a 5
```

awk

El nombre viene de sus tres creadores: Aho, Weinberger y Kernighan. Procesa y filtra texto por columnas.

```
awk '{print $1}' archivo.txt          # primera columna
awk '{print $1, $3}' archivo.txt      # columnas 1 y 3
awk -F: '{print $1}' /etc/passwd     # separador :
awk -F, '{print $2}' archivo.csv     # separador ,
awk '$3 > 100 {print $0}' archivo.txt # condición
awk '{suma += $1} END {print suma}'   # suma columna 1

# Variables: $0 línea completa, NR num línea, NF num columnas
```

sort — Sort

Ordena las líneas de un archivo o salida de texto. Por defecto ordena alfabéticamente.

```
#sort
sort archivo.txt                    # orden alfabético
sort -r archivo.txt                 # orden inverso
sort -n archivo.txt                 # orden numérico (no alfabético)
sort -u archivo.txt                 # elimina duplicados al ordenar
sort -k2 archivo.txt                # ordena por la columna 2
sort -t: -k3 -n /etc/passwd        # ordena por columna 3 usando : como separador

# Combinado con uniq para ver frecuencia
sort archivo.txt | uniq -c | sort -rn
```

uniq — Unique

Elimina o cuenta líneas duplicadas consecutivas. Requiere que el archivo esté ordenado previamente, por eso casi siempre se combina con sort.

```
#uniq
uniq archivo.txt          # elimina duplicados consecutivos
uniq -c archivo.txt       # cuenta ocurrencias de cada línea
uniq -d archivo.txt       # muestra solo las líneas duplicadas
uniq -u archivo.txt       # muestra solo las líneas únicas

# Flujo típico: ordenar primero, luego eliminar duplicados
sort archivo.txt | uniq
sort archivo.txt | uniq -c | sort -rn    # ordenado por frecuencia
```

wc — Word Count

Cuenta líneas, palabras o caracteres de un archivo o salida.

```
#wc
wc archivo.txt            # muestra líneas, palabras y bytes
wc -l archivo.txt         # solo número de líneas
wc -w archivo.txt         # solo número de palabras
wc -c archivo.txt         # solo número de bytes
wc -m archivo.txt         # solo número de caracteres

# Con tuberías
ls | wc -l                # cuántos archivos hay en el directorio
cat /etc/passwd | wc -l   # cuántos usuarios hay en el sistema
```

tee — Tee

Recibe stdout y lo envía a dos destinos al mismo tiempo: la pantalla y un archivo. El nombre viene de la tubería en forma de T. Útil cuando quieres ver la salida en tiempo real y guardarla a la vez.

```
#tee
comando | tee archivo.txt    # muestra y guarda (sobreescribe)
comando | tee -a archivo.txt # muestra y agrega al archivo
comando | tee archivo.txt | grep "patron" # guarda todo y filtra lo que ves
```

xargs — Extended Arguments

```
# xargs
cat lista.txt | xargs rm                # elimina los archivos de la lista
find . -name "*.txt" | xargs grep "patron" # busca patrón en cada archivo
encontrado
echo "uno dos tres" | xargs -n1 echo    # imprime uno por línea
find . -name "*.sh" | xargs chmod +x    # da permisos a todos los
encontrados
```

history — History

Guarda un registro de todos los comandos ejecutados en la terminal. Permite reutilizarlos sin tener que escribirlos de nuevo.

```
history                # muestra el historial completo con números
history 20             # muestra los últimos 20 comandos
history | grep chmod   # busca un comando específico en el historial
!150                   # ejecuta el comando número 150
!!                     # ejecuta el último comando ejecutado
!grep                  # ejecuta el último comando que empieza con grep
Ctrl + R               # búsqueda inversa: escribe y encuentra el comando
history -c             # limpia el historial
```

El historial se guarda en `~/.bash_history` al cerrar la sesión.

alias

Crea atajos para comandos largos o que usas frecuentemente. Un alias es simplemente un nombre corto que sustituye a un comando más largo.

```
# Crear alias
alias ll="ls -la"
alias gs="git status"
alias update="sudo apt update && sudo apt upgrade"
alias cls="clear"

# Ver todos los alias activos
alias

# Eliminar un alias
unalias ll

# Ver a qué comando apunta un alias
type ll

# Hacer el alias permanente (agregar en ~/.bashrc)
echo 'alias ll="ls -la"' >> ~/.bashrc
source ~/.bashrc
```

Los alias definidos en terminal solo duran la sesión actual. Para que persistan deben estar en `~/.bashrc`.

Expresiones Regulares

Las expresiones regulares (regex) son patrones que permiten buscar, validar y manipular texto. En Linux se usan principalmente con `grep`, `sed`, `awk` y dentro de scripts `bash`.

Tipos

Tipo	Comando	Descripción
BRE	<code>grep</code> , <code>sed</code>	Basic Regular Expressions, por defecto
ERE	<code>grep -E</code> , <code>sed -E</code> , <code>awk</code>	Extended Regular Expressions, más potente
PCRE	<code>grep -P</code>	Perl Compatible, el más completo

Metacaracteres

Símbolo	Descripción	Ejemplo	Coincide con
<code>.</code>	Cualquier carácter (excepto newline)	<code>a.c</code>	<code>abc</code> , <code>a1c</code> , <code>a-c</code>
<code>*</code>	0 o más del anterior	<code>ab*c</code>	<code>ac</code> , <code>abc</code> , <code>abbc</code>
<code>+</code>	1 o más (ERE)	<code>ab+c</code>	<code>abc</code> , <code>abbc</code>
<code>?</code>	0 o 1 (ERE)	<code>ab?c</code>	<code>ac</code> , <code>abc</code>
<code>^</code>	Inicio de línea	<code>^hola</code>	líneas que empiezan con <code>hola</code>
<code>\$</code>	Fin de línea	<code>hola\$</code>	líneas que terminan con <code>hola</code>
<code>\</code>	Escapa un metacarácter	<code>\.</code>	punto literal

Clases y cuantificadores

```
[aeiou]      # cualquiera de estos
[^0-9]       # cualquiera excepto dígitos
[a-zA-Z]     # cualquier letra

[:alpha:]    # letras
[:digit:]    # dígitos
[:space:]    # espacios
```

```
{n}           # exactamente n veces
{n,}          # n o más veces
{n,m}         # entre n y m veces
```

Ejemplos prácticos

```
# Validar email
grep -E '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$' arch.txt

# Validar IP
grep -E '^([0-9]{1,3}\.){3}[0-9]{1,3}$' arch.txt

# Líneas vacías / no vacías
grep '^$' archivo.txt
grep -v '^$' archivo.txt

# Validar en bash con [[ =~ ]]
if [[ $valor =~ ^[0-9]+$ ]]; then
    echo "es un entero"
fi
```

Referencia rápida

Símbolo	Descripción
<code>^</code>	Inicio de línea
<code>\$</code>	Fin de línea
<code>.</code>	Cualquier carácter
<code>*</code>	0 o más
<code>+</code>	1 o más
<code>?</code>	0 o 1
<code>{n,m}</code>	Entre n y m veces
<code>[abc]</code>	Uno de estos caracteres
<code>[^abc]</code>	Ninguno de estos
<code>(abc)</code>	Grupo
<code>a b</code>	a o b (alternancia)
<code>\w</code>	Letra, dígito o <code>_</code>

Símbolo	Descripción
\d	Dígito
\s	Espacio
\b	Límite de palabra

```
* /15 * * * * /scripts/monitoreo.sh      # cada 15 minutos
0 9 * * 1-5 /scripts/trabajo.sh          # lunes a viernes a las 9 AM

@reboot /scripts/inicio.sh                # al iniciar el sistema
@daily   /scripts/backup.sh               # cada día a medianoche
```

Ejemplo real completo

Script de backup que cron ejecuta cada noche a las 3:00 AM:

```
#!/bin/bash
# backup.sh - script de backup diario
set -euo pipefail

ORIGEN="/home/francisco"
DESTINO="/backups"
FECHA=$(date +%Y-%m-%d)
ARCHIVO="backup_${FECHA}.tar.gz"
LOG="/var/log/backup.log"

mkdir -p $DESTINO
echo "[$FECHA] Iniciando backup..." >> $LOG
tar -czf $DESTINO/$ARCHIVO $ORIGEN >> $LOG 2>&1

if [ $? -eq 0 ]; then
    echo "[$FECHA] OK: $ARCHIVO" >> $LOG
else
    echo "[$FECHA] ERROR" >> $LOG; exit 1
fi

find $DESTINO -name "backup_*.tar.gz" -mtime +7 -exec rm {} \;
echo "[$FECHA] Backups viejos eliminados" >> $LOG

# Entrada en crontab:
# 0 3 * * * /scripts/backup.sh >> /var/log/backup.log 2>&1
```

cron vs crontab

Son parte del mismo sistema pero cumplen roles distintos:

cron es el servicio (demonio) del sistema. Corre en segundo plano y ejecuta las tareas. Es el motor.

crontab es el archivo donde se definen las tareas, y tambien el comando para editarlo. Es la configuracion.

Analogia: cron es el motor del auto, crontab es el panel donde defines que hacer y cuando.

Cada usuario tiene su propio crontab. El sistema tambien tiene /etc/crontab y directorios como /etc/cron.daily/ que cron lee automaticamente.

at

Ejecuta una tarea una sola vez en un momento futuro.

```
sudo apt install at && sudo systemctl enable --now atd

at 15:30                # a las 15:30 de hoy
at now + 2 hours
at now + 30 minutes

# Prompt interactivo
at 15:30
at> /scripts/tarea.sh
at> <Ctrl+D>

atq                    # ver tareas pendientes
atrm 2                 # eliminar tarea número 2
```

systemd timers

Alternativa moderna a cron, integrada con systemd. Más robusta y con mejor manejo de logs. Requiere dos archivos por tarea: .service (qué ejecutar) y .timer (cuándo).

```
# /etc/systemd/system/backup.service
[Unit]
Description=Script de backup
[Service]
Type=oneshot
ExecStart=/scripts/backup.sh
User=francisco
```

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Backup diario
[Timer]
OnCalendar=*-*-* 03:00:00
Persistent=true
[Install]
WantedBy=timers.target

# Activar
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers
journalctl -u backup.service      # ver logs
```

Comparativa

	cron	at	systemd timers
Tareas periódicas	✓	✗	✓
Tarea única	✗	✓	✓
Logs integrados	✗	✗	✓
Recupera tareas perdidas	✗	✗	✓ (Persistent=true)
Complejidad	Baja	Baja	Media
Disponibilidad	Universal	Universal	Sistemas con systemd

— Fin del Tomo I —